

Parkour entre azoteas

Entrega individual de Alberto | Entorno, física, cámara y apariencia

SKYLINE transforma el laboratorio inicial en un circuito de **16 azoteas flotantes** sobre una ciudad al atardecer. El objetivo es llegar a la meta; se pueden recoger 15 cristales y guardar avance en tres puntos de control intermedios.



Captura 1. Inicio sobre una plataforma con colisión, personaje en Idle y HUD de tiempo, estado y progreso.

Controles y objetivo

Acción	Control
Moverse / saltar	WASD / Espacio (solo con apoyo)
Cámara / cursor	Ratón o flechas / Esc libera; clic captura
Reintentar / rendimiento	R vuelve al checkpoint; en meta reinicia / F: límite 30 o 144 FPS

La ciudad, árboles y accesorios son decorativos. Las azoteas transitables tienen colisiones de caja o cilindro. Cielo procedural, iluminación cálida y sombras definen el entorno. El récord se guarda localmente; las caídas mantienen el cronómetro.

Arquitectura y movimiento

Separación de física, presentación y reglas de la partida

Escena / archivo	Responsabilidad
parkour.tscn + parkour.gd	Nivel editable, aterrizajes, checkpoints, cristales, tiempo y meta.
courier_player.tscn + player.gd	CharacterBody3D, cápsula, movimiento y FSM.
courier_visual.tscn + courier_visual.gd	Modelo nativo independiente; pivotes de extremidades y bufanda.
CameraRig / Pitch / SpringArm3D	Yaw, inclinación, seguimiento y prueba de obstáculos.
crystal.gd / parkour_hud.gd	Recogida por Area3D / interfaz y estado activo.

Jerarquía editable del personaje

Player (CharacterBody3D)

CollisionShape3D: cápsula, altura 1.8 m y radio 0.42 m

Visual: instancia de courier_visual.tscn

Body: casco, torso, mochila y pivotes de brazos/piernas

CameraRig / Pitch / SpringArm3D / Camera3D

Dónde se utiliza delta

En `_physics_process(delta)`, a 60 Hz, `Input.get_vector` limita la diagonal. La base de yaw de la cámara transforma la entrada al plano XZ. Mirar arriba o abajo no altera la rapidez horizontal.

Velocidad: 6 m/s. **Aceleración:** 22 m/s². **Gravedad:** 22 m/s². La aceleración y gravedad se multiplican por delta una sola vez para modificar velocity. El salto asigna directamente **9 m/s** al componente vertical cuando hay suelo.

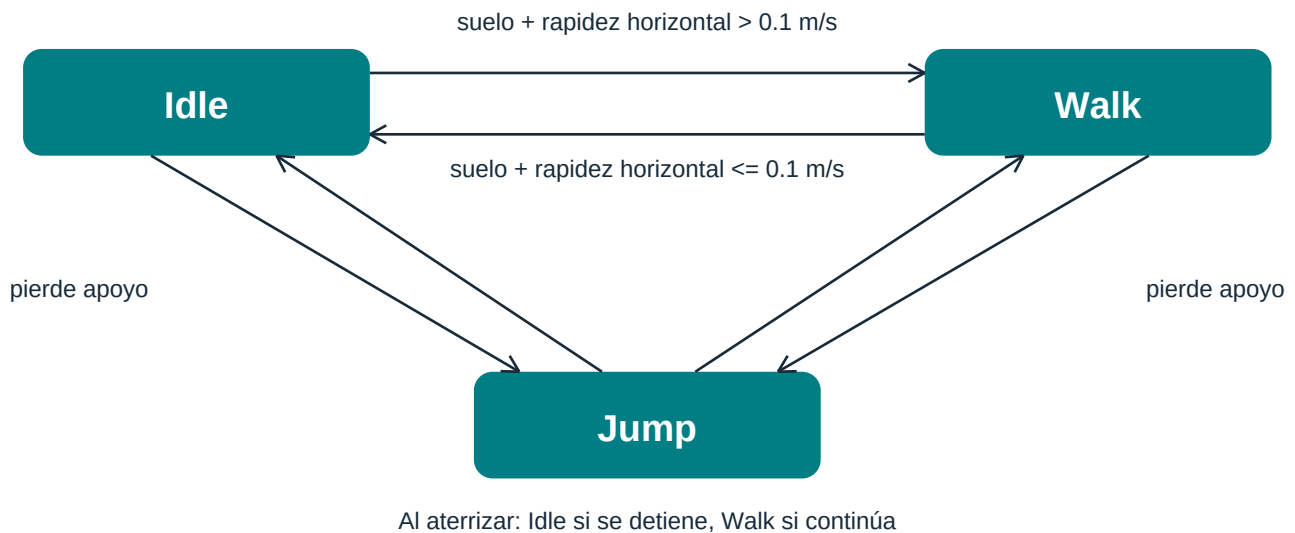
`move_and_slide()` integra velocity usando el paso físico; multiplicar antes esa velocidad por delta aplicaría el tiempo dos veces. En `_process(delta)` solo se actualizan orientación y poses visuales. Las flechas usan delta para girar por segundo; el ratón ya proporciona desplazamiento acumulado.

La señal `physics_stepped` se emite después de mover y actualizar la FSM. El nivel examina esos contactos actualizados para guardar el checkpoint, evitando reutilizar el suelo de una posición anterior tras reaparecer.

FSM y cámara

Estados explícitos, seguimiento y recuperación de caídas

Diagrama de transiciones



Jump incluye tanto ascenso como caída. Las transiciones se deciden después de `move_and_slide()` y se validan con una tabla de cambios legales. El estado aparece en el HUD y en el registro FSM. Las poses de brazos y piernas acompañan el estado; no controlan la física.

Cámara orbital nativa

El `SpringArm3D` de SKYLINE mide **6 m**, usa una esfera de 0.22 m y margen de 0.15 m. Su máscara consulta el nivel y excluye el RID del personaje. Pitch se limita entre -1.1 y 0.25 rad; FOV es 72° y el plano cercano 0.08 m.

En la prueba con una pared temporal, el brazo se retrajo a **1.078 m** y un giro completo de 360° mantuvo la cámara del lado interior. Se utilizó el controlador actual. Los edificios y portales decorativos no son paredes físicas.

Puntos de control y meta

Al aterrizar en las azoteas 5, 9 y 13 se guarda la posición de regreso. Si el personaje cae por debajo de $Y = -12$ m, se reinician posición, velocidad e interpolación. Llegar a la azotea 16 finaliza el tiempo. Los cristales son opcionales para completar el recorrido.

Personaje y procedencia

Mensajero original de práctica | Modelo nativo, sin servicio 3D externo

Alberto autorizó el recurso original de práctica. SKYLINE utiliza **primitivas de Godot generadas por Python**, con asistencia de Codex. No se usó Tripo/Meshy. El modelo actual es .tscn; el glTF del robot anterior se conserva como comparación. Ambos separan apariencia y controlador.

Ficha medida en reposo	Resultado
Piezas / triángulos / vértices	30 / 23,112 / 13,516 (sumados por instancia)
Materiales / texturas / UV	5 PBR / 0 imágenes / 30 superficies con UV nativas
Ancho × alto × fondo	0.998 × 1.867 × 0.973 m, incluidos mochila y bufanda
Escala / orientación	Envolvente 1,1,1; una unidad = un metro; frente -Z, arriba +Y
Huesos / pesos / clips	0; animación de piezas rígidas mediante pivotes

Antes y después

Criterio	Robot anterior	Mensajero actual
Forma y detalle	11 cajas; 132 triángulos	30 piezas redondeadas; 23,112 triángulos
Materiales y movimiento	3; oscilación global	5; extremidades, torso y bufanda
Resultado	Prueba funcional	Mayor identidad visual y mayor costo geométrico

Verificaciones y límites declarados

- 1. Malla y normales:** vectores finitos y unitarios. Se detectaron **928 triángulos de área casi nula** en las primitivas, incluidos en el total; no se hizo reparación topológica ni se afirma una malla optimizada.
- 2. Escala:** altura visual 1.867 m y cápsula 1.8 m; casco y accesorios sobresalen ligeramente de la colisión.
- 3. Materiales y articulaciones:** cinco materiales incluidos, sin imágenes faltantes; no hay rig ni skinning. La deformación corresponde a Tarea 2.

Fecha: 06.10.2026. Arte original CC0. Generador: tools/build_parkour.py; semilla local de ciudad 17. Solicitud exacta, dirección aplicada y bitácora en Prompts/SKYLINE_*. Ficha completa y hash en assets/parkour/FICHA.md.

Evidencias de ejecución

Capturas reales del motor y mediciones del controlador actual



Captura 2. Salto del mensajero entre plataformas. El estado Jump se muestra en el HUD. La prueba técnica verifica que pulsar Espacio de nuevo en el aire no genera otro salto.



Captura 3. Aterrizaje en el checkpoint de la azotea 9. La prueba fuerza una caída y comprueba el regreso al punto guardado; después completa el recorrido y recoge 15 cristales.

Comparación con dos límites de render

Intervalo	30 FPS	144 FPS
120 pasos físicos = 2 s	11.23112 m	11.23112 m
FPS observados en la medición	30	144

Diferencia: **0.00000 m** en esta ejecución. La distancia es menor que 12 m por la aceleración desde reposo. La física permanece a 60 Hz; el render no cambia ese paso. La medición se realizó con render normal sobre un suelo temporal amplio, no con el grabador de video. Otros equipos pueden presentar pequeñas diferencias de muestreo.

Reproducción y entrega

Proyecto completo, video y registro de pruebas

Abrir una copia del proyecto

Extraer Skyline_Alberto.zip, importar project.godot con Godot 4.7.2 y esperar la importación. **F5** inicia SKYLINE; abrir parkour.tscn, courier_player.tscn o courier_visual.tscn permite editar sus nodos en la vista 3D. Compatibility/OpenGL y GodotPhysics3D no requieren plugins.

Resultados y cobertura

Prueba	Resultado registrado
Recorrido actual	20 comprobaciones: inicio, 15 saltos, checkpoint intermedio, meta, cristales y regreso tras caída. Sin fallos.
Auditoría actual	11 comprobaciones: 30/144 FPS, Idle/Walk/Jump, doble salto, dirección de cámara y giro junto a pared. Sin fallos.
Copia limpia	Reimportación sin .godot y ejecución del parkour y laboratorio de regresión. Sin errores registrados.

Las pruebas son automatizadas con el controlador real. El video es una demostración automatizada de aproximadamente un minuto, con pausas para observar personaje, checkpoints y meta; no es una partida manual ni una medición de rendimiento.

Archivos de entrega

Proyecto completo: Skyline_Alberto.zip

Video: Skyline_Alberto.mp4

README.md: controles, versión, escenas, pruebas e instrucciones.

Prompts/SKYLINE_*: solicitud, bitácora y comparación.

assets/parkour/FICHA.md: malla, materiales, escala y procedencia.

Enlaces locales: conservar PDF en output/pdf, video en output/evidence y ZIP en output. Si la institución exige URLs web, subir ZIP y video y sustituir estos enlaces. No se proporcionó destino de publicación.

Cómo repetir y consultar

```
godot --path . -- --parkour-test
```

```
godot --path . --script tests/skyline_audit.gd -- --test
```

Evidencia en output/evidence: parkour_results.json, skyline_audit.json y skyline_clean_copy.json.

Referencias técnicas: [CharacterBody3D](#) y [move_and_slide\(\)](#) y [SpringArm3D](#) (documentación oficial). Los materiales de apoyo internos del curso no se proporcionaron; no se atribuye revisión de su contenido.